



AUDIT REPORT

September 2026

For



Hashlock Market

Table of Content

Executive Summary	04
Number of Security Issues per Severity	06
Summary of Issues	07
Checked Vulnerabilities	09
Techniques and Methods	11
Types of Severity	13
Types of Issues	14
Severity Matrix	15
Impact	15
Likelihood	15
■ Critical Severity Issues	16
1. Taker steals ETH leg via unvalidated TRON timelock in onTronFunded	16
■ High Severity Issues	18
2. setSwapAddress allows post-funding payout edits, silently killing the secret relay	18
3. Constant-fee BTC claim loses the CLTV race, forfeiting the leg	20
4. Enabling protocol fees permanently strands the taker's Bitcoin escrow on any BTC-funded leg, because no shipped client pays the fee inside the funding transaction the server requires	22
5. Unverified EVM clone address lets a decoy rebind the escrow	24
6. Protocol fee is bypassable to zero on EVM and TRON legs	26
■ Medium Severity Issues	28

7. Zero-amount fund squats swapId and permanently blocks the leg	28
8. Expired RFQs stay fillable, handing makers a free perpetual option	30
9. Dust output consumes the BTC dedup slot and strands the swap	32
10. Claim and refund sweep only the largest UTXO, silently abandoning the rest of a multi-UTXO escrow	34
■ Low Severity Issues	36
11. _fund escrows unverified amount, permanently locking fee-on-transfer swaps	36
12. encodeScriptNumber truncates post-2038 timelocks, voiding the CLTV refund guard	38
13. Floating Solidity Compiler Pragmas Can Introduce Build and Deployment Risk	40
■ Informational Severity Issues	41
14. USDT Issuer-Level Blacklisting / Freezing	41
15. No Recovery Mechanism for Accidentally Sent or Stuck Tokens	42
16. SharedHTLC Can Over-Credit Swaps With Hook/Rebasing Tokens	43
17. Missing recipient validation lets a mis-set swap leg burn or strand its escrow	44
Functional Tests	45
Threat Model	52
1. External Dependencies & Trust Boundaries	52
2. Entry & Exit Points Analysis (Function-Level)	56
2.1 Function Entry / Exit Table	56
3. Asset Flow Mapping (Critical Contracts)	67
3.1 Asset-Holding Contracts	67
3.2 Asset Entry & Exit Functions	67
Automated Tests	70
Closing Summary & Disclaimer	71

Executive Summary

Project Name HashLock

Protocol Type Non-custodial P2P cross-chain OTC / atomic swap protocol

Project URL <https://hashlock.markets>

Overview Hashlock Markets is a sealed-bid RFQ-based cross-chain trading protocol that enables non-custodial atomic swaps between counterparties using Hash Time-Locked Contracts (HTLCs). The reviewed implementation includes EVM HTLC contracts, TRON HTLC infrastructure, Bitcoin P2WSH HTLC scripts, and an off-chain RFQ, settlement, watcher, and keeper infrastructure.

Audit Scope The scope of this Audit was to analyze the HashLock Smart Contracts for quality, security, and correctness.

Source Code Link <https://github.com/Hashlock-Tech>

Contracts in Scope Smart Contracts

- HTLCFactory.sol
- HTLCImplementation.sol
- SharedHTLC.sol

Bitcoin Protocol Logic Review

- packages/api/src/chains/bitcoin/*

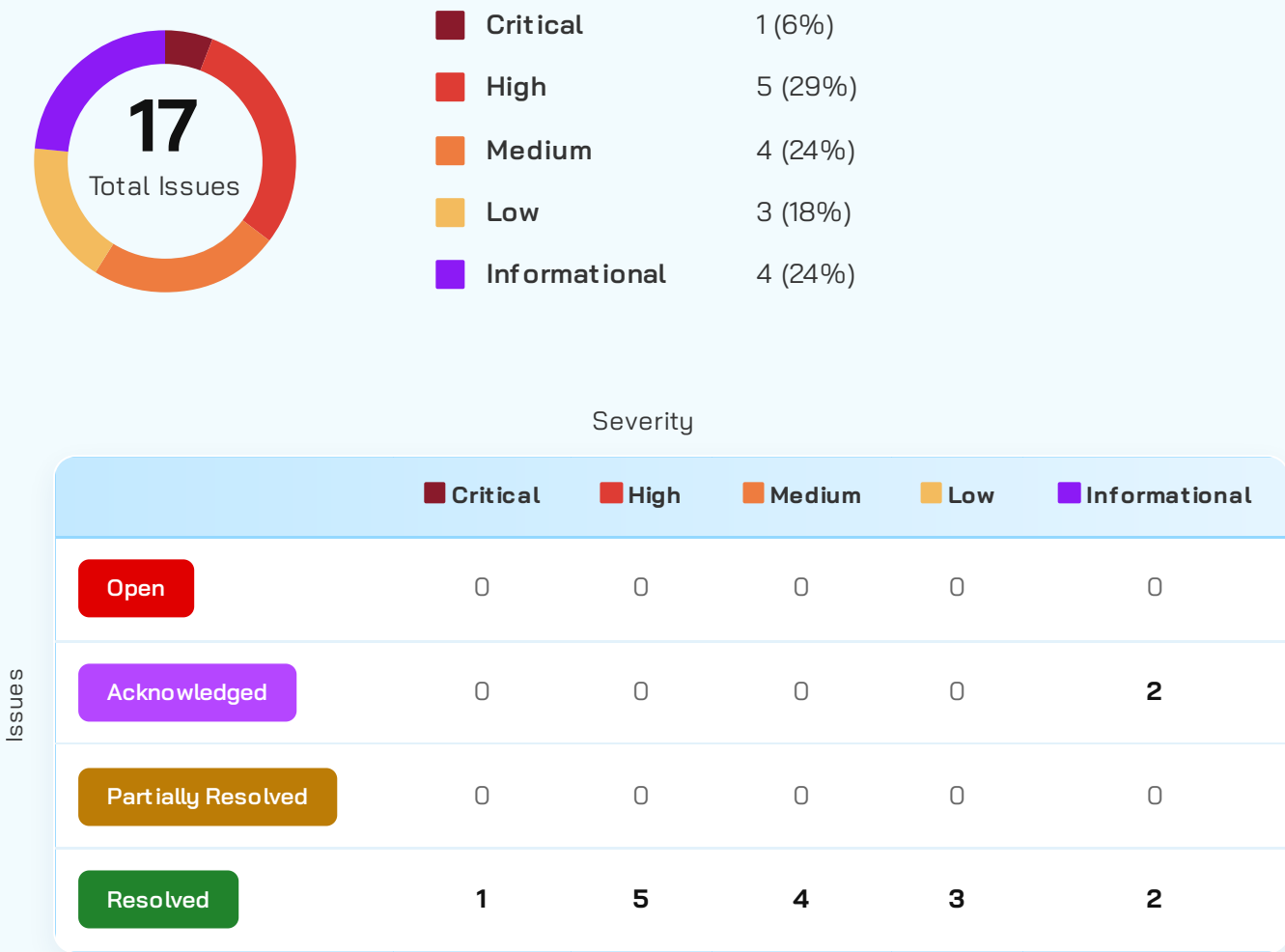
Settlement Logic Review

- packages/api/src/settlement/*
- chains/evm/*
- chains/tron/*
- chains/timelock.ts
- rfq/service.ts

Branch	audit
Commit Hash	e8f4ba564b970394155f5037ff649ae1db5274ae
Language	Solidity, TypeScript
Blockchain	Ethereum / EVM-compatible chains, TRON, Bitcoin
Method	Manual Analysis, Functional Testing, Automated Testing
Review 1	6th August 2026 - 19th August 2026
Updated Code Received	21st August 2026
Review 2	31st August 2026 - 28th September 2026
Fixed In	b8a1a9aabaff34d9b8b97e26b9c3825c285b8bd1

✔ Verify the Authenticity of Report on QuillAudits Leaderboard:
<https://www.quillaudits.com/leaderboard>

Number of Issues per Severity



Summary of Issues

Issue No.	Issue Title	Severity	Status
1	Taker steals ETH leg via unvalidated TRON timelock in onTronFunded	Critical	Resolved
2	setSwapAddress allows post-funding payout edits, silently killing the secret relay	High	Resolved
3	Constant-fee BTC claim loses the CLTV race, forfeiting the leg	High	Resolved
4	Enabling protocol fees permanently strands the taker's Bitcoin escrow on any BTC-funded leg	High	Resolved
5	Unverified EVM clone address lets a decoy rebind the escrow	High	Resolved
6	Protocol fee is bypassable to zero on EVM and TRON legs	High	Resolved
7	Zero-amount fund squats swapId and permanently blocks the leg	Medium	Resolved
8	Expired RFQs stay fillable, handing makers a free perpetual option	Medium	Resolved
9	Dust output consumes the BTC dedup slot and strands the swap	Medium	Resolved

Issue No.	Issue Title	Severity	Status
10	Claim and refund sweep only the largest UTXO, silently abandoning the rest of a multi-UTXO escrow	Medium	Resolved
11	_fund escrows unverified amount, permanently locking fee-on-transfer swaps	Low	Resolved
12	encodeScriptNumber truncates post-2038 timelocks, voiding the CLTV refund guard	Low	Resolved
13	Floating Solidity Compiler Pragmas Can Introduce Build and Deployment Risk	Low	Resolved
14	USDT Issuer-Level Blacklisting / Freezing	Informational	Acknowledged
15	No Recovery Mechanism for Accidentally Sent or Stuck Tokens	Informational	Acknowledged
16	SharedHTLC Can Over-Credit Swaps With Hook/Rebasing Tokens	Informational	Resolved
17	Missing recipient validation lets a mis-set swap leg burn or strand its escrow	Informational	Resolved

Checked Vulnerabilities

✓ Access Management

✓ Arbitrary write to storage

✓ Centralization of control

✓ Ether theft

✓ Improper or missing events

✓ Logical issues and flaws

✓ Arithmetic Computations Correctness

✓ Race conditions/front running

✓ SWC Registry

✓ Re-entrancy

✓ Timestamp Dependence

✓ Gas Limit and Loops

✓ Exception Disorder

✓ Gasless Send

✓ Use of tx.origin

✓ Malicious libraries

✓ Compiler version not fixed

✓ Address hardcoded

✓ Divide before multiply

✓ Integer overflow/underflow

✓ ERC's conformance

✓ Dangerous strict equalities

✓ Tautology or contradiction

✓ Return values of low-level calls

✓ Missing Zero Address Validation

✓ Private modifier

✓ Revert/require functions

✓ Multiple Sends

✓ Using suicide

✓ Using delegatecall

✓ Upgradeable safety

✓ Style guide violation

✓ Using throw

✓ Unsafe type inference

✓ Using inline assembly

✓ Implicit visibility level

Techniques and Methods

Throughout the audit of smart contracts, care was taken to ensure:

- The overall quality of code.
- Use of best practices.
- Code documentation and comments, match logic and expected behavior.
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper.
- Implementation of ERC standards.
- Efficient use of gas.
- Code is safe from re-entrancy and other vulnerabilities.

The following techniques, methods, and tools were used to review all the smart contracts:

Structural Analysis

In this step, we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.

Static Analysis

A static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.

Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

Gas Consumption

In this step, we have checked the behavior of smart contracts in production. Checks were done to know how much gas gets consumed and the possibilities of optimization of code to reduce gas consumption.

Tools and Platforms Used for Audit

Remix IDE, Foundry, Solhint, Mythril, Slither, Solidity Static Analysis.

Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

■ Critical: Immediate and Catastrophic Impact

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

■ High (H): Significant Risk of Major Loss or Compromise

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

■ Medium (M): Potential for Moderate Harm Under Specific Circumstances

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

■ Low (L): Minor Imperfections with Limited Repercussions

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

■ Informational (I): Opportunities for Improvement, Not Immediate Risks

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.

Types of Issues

Open Security vulnerabilities identified that must be resolved and are currently unresolved.	Resolved These are the issues identified in the initial audit and have been successfully fixed.
Acknowledged Vulnerabilities which have been acknowledged but are yet to be resolved.	Partially Resolved Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

Severity Matrix

		Impact		
		 High	 Medium	 Low
Likelihood	 High	Critical	High	Medium
	 Medium	High	Medium	Low
	 Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely.
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.

Critical Severity Issues

Taker steals ETH leg via unvalidated TRON timelock in onTronFunded

Resolved

Path

`watcher.ts`

Function Name

`onTronFunded`

Description

Atomic-swap safety here rests entirely on role-based timelock asymmetry. `resolveLegRoles` assigns the longer timelock to the initiator, the secret holder, so that after the initiator reveals by claiming the participant's leg, the participant still has at least `MIN_TIMELOCK_GAP_SECONDS` to claim the initiator's leg before it can be refunded. For an ETH↔TRON pair the two chains carry equal base tiers of 14400 seconds, so the tie in `resolveLegRoles` sends the long leg to leg B, the taker: TRON at 21600 seconds and ETH at 14400. Each watcher is supposed to bind the on-chain funding to the agreed terms before the counterparty is signalled to fund their side.

When the taker funds `SharedHTLC.fund`, they choose timelock freely, because the contract enforces only the `MIN_TIMELOCK` floor. `onTronFunded` then verifies the recipient and the amount against the agreed terms, but never compares the on-chain timelock to `legTimelockSec`, and the watcher cannot supply it anyway: `SharedHTLC` emits timelock as the sixth `Funded` argument while `startTronWatcher` decodes only the `swapId`, recipient, amount and hashlock positions, so `TronFunded` carries no timelock field at all. The handler then publishes `leg_funded` unconditionally, and the maker funds their honest 4h ETH leg. At T+30 minutes the taker calls `SharedHTLC.refund` and recovers their TRON without revealing anything, then claims the maker's ETH with the secret; the maker's TRON claim reverts `AlreadySettled`. This breaks the invariant `resolveLegRoles` exists to establish, that the initiator's leg expires strictly after the participant's. The omission is unambiguous rather than intentional: the sibling handler `onEvmSwapFunded` performs exactly this timelock comparison against `legTimelockSec` before recording its leg.

Taker steals ETH leg via unvalidated TRON timelock in onTronFunded

(continued)

The bug applies precisely when leg A is Ethereum and leg B is TRON, which is what `resolveLegRoles` produces for that pair. It does not apply when TRON is leg A, because the tie then places the long leg on Ethereum, where `onEvmSwapFunded` validates the timelock. It does not apply to BTC↔TRON, because Bitcoin's 86400-second tier always takes the long leg. It requires no privilege, no keeper or operator compromise, no race, and no capital beyond the taker's own leg for 30 minutes.

Impact

The taker refunds their TRON leg at the 30-minute mark and then claims the maker's ETH with the secret, an irreversible one-sided loss of the maker's entire principal with zero attacker capital at risk.

Likelihood

High

Recommendation

Decode timelock from the Funded event in `startTronWatcher` and reject any funding whose timelock differs from `legTimelockSec`, matching the check `onEvmSwapFunded` already performs.

Enforce the asymmetry independently of any single handler by withholding the `leg_funded` signal to the participant until the initiator's leg is confirmed to expire at least `MIN_TIMELOCK_GAP_SECONDS` after the participant's.

High Severity Issues

setSwapAddress allows post-funding payout edits, silently killing the secret relay

Resolved

Path

`service.ts`

Function Name

`setSwapAddress`

Description

Every funded event is bound to the swap's agreed terms before the platform will act on it, because the on-chain contracts are permissionless and matching on the shared hashlock alone would let an attacker register a spoofed escrow as the victim's leg. `onEvmSwapFunded` therefore compares the clone's immutable recipient, amount, token and timelock against the swap record and refuses to advance on any mismatch. `setSwapAddress` exists so a party can supply their receive or refund address from a connected wallet when the address book was empty at agreement time.

That binding compares an on-chain immutable against a mutable database row, and `setSwapAddress` gates only on party membership: there is no status check and no already-funded check, so the maker can rewrite `bPayoutAddress` at any point in the swap's life. Rewriting it after the taker's funding transaction is built does not move the clone's recipient, which is fixed in the CWIA args, but it changes the value the watcher compares against. `onEvmSwapFunded` then takes the mismatch branch and returns before writing `bHtlcAddress`, while the watcher has already added the clone to its in-memory tracked set and advances its durable cursor past that block range regardless of the handler's outcome. `onEvmSwapFunded` is the only writer of that column for an EVM leg, so it stays NULL permanently and the swap status never advances. When the maker claims and `Claimed` fires, `onSecretRevealed` calls `findByHtlcAddress`, matches no row, and returns silently: no secret stored, no counter-leg relay, no status publish, no notification. The property that breaks is that a leg's agreed terms must be immutable once an escrow has been funded against them, and here one party can retroactively invalidate the reference the platform verifies against.

setSwapAddress allows post-funding payout edits, silently killing the secret relay *(continued)*

The window is generous rather than tight, since the EVM watcher waits five confirmations before processing SwapCreated, and the maker needs only one authenticated call inside it. In-memory tracking is the sole thing keeping the clone observed at all: evmTrackedClones re-seeds the watch set from the same NULL column, so any API restart drops the clone entirely and the reveal is missed even by a fresh process. The vector is not EVM-specific, because onTronFunded binds against the same mutable legPayout value; the BTC leg is unaffected only because onBtcFunded never compares a payout address.

Impact

The maker claims the taker's entire ETH leg while the platform stores and relays nothing, then refunds their own BTC once the 24h CLTV lapses, taking both legs from a taker who is never told the secret was revealed.

Likelihood

High

Recommendation

Reject setSwapAddress for any leg whose escrow is already recorded or whose swap has advanced past agreement, so the terms a funded event is verified against cannot change after funding.

Snapshot the agreed payout, amount, token and timelock into immutable per-leg columns at agreement time and bind the watcher checks to that snapshot rather than to the mutable address fields.

High Severity Issues

Constant-fee BTC claim loses the CLTV race, forfeiting the leg

Resolved

Path

```
btc-privy.ts  
btc-claim.ts
```

Function Name

```
claimBtc and refundBtc; btcClaimViaPrivy and btcRefundViaPrivy
```

Description

The participant's BTC claim and the initiator's CLTV refund spend the same UTXO, so confirmation latency is the security property, not a cost setting. Both clients pay an absolute constant of 600 satoshis instead of a rate, and no caller ever overrides it. Since the transactions have deterministic sizes, that fixes the rate: 4.32 sat/vB on the 139 vB claim and 4.62 on the 130 vB refund. Nothing in the repository queries a fee estimate. Neither transaction can be repaired after broadcast either. The claim sets no sequence, so the library default leaves it final and non-replaceable, and the refund's 0xffffffff sits one above the BIP-125 threshold, so it is non-replaceable too. There is no bump action, no rebroadcast and no retry.

The initiator controls the reveal and knows the deadline, since the API serves the leg timelock. They open a swap while the network already clears above 4.32 sat/vB, claim the EVM leg to take the participant's stablecoin and publish the secret, and wait. The participant's claim underpays and sits unconfirmed, and may be evicted outright once the mempool passes its 300 MB cap, with nothing to rebroadcast it. The refund then confirms at market rate against an opponent that is stuck or gone. The participant holds roughly twenty hours between reveal and deadline, so this needs the clearing rate to hold above 4.32 sat/vB across that window; the initiator cannot manufacture that, but selecting an episode already underway costs nothing. Note the inversion: the funding transaction, which has no deadline, is priced by the user's wallet from live data, while both deadline-bound transactions are pinned to a constant.

Impact

The participant loses their entire BTC leg to a refund they cannot outbid, after having already paid out their own leg, with no bump, replace or rebroadcast path in any shipped client.

Constant-fee BTC claim loses the CLTV race, forfeiting the leg*(continued)***Likelihood**

Medium

Recommendation

Size every deadline-bound spend from Esplora's fee estimate multiplied by the transaction's known vsize, with a floor and a sanity ceiling, and scale the target rate upward as the remaining time to the CLTV shrinks.

High Severity Issues

Enabling protocol fees permanently strands the taker's Bitcoin escrow on any BTC-funded leg, because no shipped client pays the fee inside the funding transaction the server requires

Resolved

Path

```
packages/api/src/settlement/service.ts
```

Function Name

```
onBtcFunded (server); doFundBtc (web funding); fund (Telegram Mini App funding); fundBtcLeg (the correct but unused builder)
```

Description

When a leg carries the protocol fee, `onBtcFunded` requires the fee to be collected inside the funding transaction itself: if `feeAmount` is non-zero and the leg's asset is the fee asset, it calls `txPaidToAddress` against the funding txid and refuses to advance the swap unless that same transaction also pays the BTC treasury at least the fee. The fee always lands on the taker's funded leg, computed on `makerWants` at agreement time, and `resolveLegRoles` always assigns Bitcoin the long leg because its configured timelock exceeds the EVM and Tron timelocks. So every swap in which the taker sells BTC puts the fee on the Bitcoin leg and takes this path.

No shipped client produces the transaction the server demands. The web path funds the escrow, then pays the treasury in a second, separate transfer with a different txid; the Telegram Mini App computes the fee but its Bitcoin branch calls `btcSendViaPrivy` with only the escrow output and never pays the treasury at all. The one correct builder, `fundBtcLeg`, assembles escrow, treasury, and change in a single PSBT, but nothing in the monorepo calls it. The funding transaction is real and confirmed, so the taker's BTC is locked in the P2WSH, yet `onBtcFunded` finds no treasury output in that txid and never advances the swap, which stays at agreed. The address-keyed funding dedup in the BTC watcher then retires the address after that first rejected observation, so no later transaction can rescue it. The UI compounds the error by telling the user the fee is paid in one transaction.

Enabling protocol fees permanently strands the taker's Bitcoin escrow on any BTC-funded leg, because no shipped client pays the fee inside the funding transaction the server requires *(continued)*

Impact

Once fees are enabled, every taker-sells-BTC swap deterministically strands the taker's funded escrow: the swap never advances past agreed, and the on-chain BTC is recoverable only by a manual refund after the timelock, with no keeper or scheduler doing it for Bitcoin. It fails closed (no fee is skimmed and nothing is stolen), so funds are lost to availability, not to theft.

Likelihood

Medium

Recommendation

Fund every BTC path (web, Privy, Telegram) through fundBtcLeg so the escrow and treasury outputs share one transaction, and fix the "one transaction" UI copy.

If a single tx is ever infeasible, make onBtcFunded credit treasury payments cumulatively per swap while still gating advancement on both escrow and fee, so it cannot fail open.

High Severity Issues

Unverified EVM clone address lets a decoy rebind the escrow

Resolved

Path

```
service.ts  
watchers.ts
```

Function Name

```
onEvmSwapFunded
```

Description

When an EVM leg is funded, the settlement watcher observes the factory's SwapCreated log and calls onEvmSwapFunded, which records the on-chain escrow clone belonging to a swap. That swap settles later: on a Claimed event the backend looks the escrow up by address and relays the revealed secret to the counter-leg so the counterparty can claim.

onEvmSwapFunded locates the swap by hashlock and then admits the clone on four term comparisons only – recipient, amount, token, and timelock. It never recomputes the deterministic CREATE2 address the swap actually expects via HTLCFactory.computeAddress, which the contract ships for exactly this "verify before funding" purpose, and the write that records aHtlcAddress is unconditional, so a later term-matching event overwrites the escrow already recorded. Because createSwap is permissionless and all four terms are public columns on the party-gated swap row, an attacker deploys a second clone with the same hashlock and identical terms and, by having its SwapCreated ingested after the honest one, rebinds aHtlcAddress to the decoy. This breaks the invariant that a swap's recorded escrow is the unique clone its own canonical funding produces: the honest clone's later Claimed resolves through findByHtlcAddress to no row, so onSecretRevealed returns without storing the secret or invoking relayCounterLeg.

The bug applies only when the attacker is the swap initiator, which is self-selected because the initiator is the secret holder, and only to an EVM counterparty-funded leg. It does not harm a counterparty that independently watches the chain: the secret is public in the honest clone's claim calldata, so a self-monitoring counterparty reads it directly and still claims its counter-leg. The harm lands on the standard product userbase (Privy/TMA), which relies on the backend secret_available push and, for a Bitcoin counter-leg, has no keeper fallback because keeperAutoClaims('bitcoin') is false.

Unverified EVM clone address lets a decoy rebind the escrow *(continued)***Impact**

On any EVM↔BTC or EVM↔TRON swap whose EVM leg is the counterparty's, the initiator-attacker rebinds the escrow to a decoy, claims the honest clone to take 100% of the counterparty's funded leg, and – because the reveal is never relayed – leaves the counterparty unsignalled and refunds their own leg after its timelock, keeping both legs. The loss is the counterparty's entire funded amount and is irreversible for any counterparty that does not watch the chain itself. The attacker's only cost is gas plus briefly escrowing the decoy's leg amount as float, which they recover by refunding the decoy after its matched short-leg timelock; the decoy's Refunded event reaches no handler, so the refund does not un-suppress the relay.

Likelihood

Medium

Recommendation

Verify the funded clone against the CREATE2 address the swap expects, recomputed via `HTLCFactory.computeAddress` from its salt and immutables, and reject any mismatch; make the escrow write first-writer-wins. Apply the same funder-and-id binding to `onTronFunded`.

High Severity Issues

Protocol fee is bypassable to zero on EVM and TRON legs

Resolved

Path

`service.ts`

Function Name

`onEvmSwapFunded`
`onTronFunded`

Description

The protocol fee is charged in basis points of the taker's outgoing leg, assigned to leg B and recorded on the swap at agreement time via `feeAmount` and `feeAssetId`. `rfq/fee.ts` states that on-chain collection is a separate settlement concern and that the full accounting path is exercised so enabling the fee later is "a config change, not a code change." Collection is meant to happen inside the funding call, which routes the fee to the treasury.

The fee is a caller-supplied argument to the permissionless funding functions, and the contract accepts zero: `HTLCFactory.createSwap` routes the fee to the treasury only when it is non-zero, reverting only on a non-zero fee with no treasury configured, so `fee = 0` always passes; `SharedHTLC.fund` is identical. The backend cannot detect this, because the funded events omit the fee: `SwapCreated` and `Funded` carry no fee field, and the separate `FeeCollected` event fires only when the fee is non-zero and is declared in neither watcher ABI. The funded-event handlers then never inspect it — `onEvmSwapFunded` binds recipient, amount, token, and timelock only, and `onTronFunded` binds recipient and amount only — while `onBtcFunded` alone enforces the fee, requiring the funding transaction to pay the treasury at least `feeAmount` before advancing. This breaks the invariant that a settled leg has paid the agreed protocol fee: no post-fund reconciliation exists, so a fee not taken at fund time is never taken.

The bug is dormant under the current default `PROTOCOL_FEE_BPS = 0`, where the fee is always zero and there is nothing to bypass. It becomes live the instant fees are enabled, which is a config change with no code deploy, and applies to every swap whose fee-bearing leg is on EVM or TRON — that is, all fee-bearing pairs except taker-pays-BTC. It requires only that the taker submit their own funding transaction rather than the app-built one, which supplies the correct fee.

Protocol fee is bypassable to zero on EVM and TRON legs *(continued)***Impact**

Once fees are enabled, any taker on any EVM/TRON funding leg signs their own createSwap or fund call with a zero fee, produces an escrow that satisfies every check the watcher performs, and settles a fully valid swap having paid the treasury nothing. This is a complete and permanent defeat of the protocol's fee-revenue stream on the majority of fee-bearing swaps, at no cost to the taker beyond the gas they would pay anyway, and it is unrecoverable: the fee is moved once at fund time and never reconciled at settlement, so revenue foregone on a swap cannot be recaptured. No user funds are at risk, since the fee sits on top of the escrowed amount; the damage is entirely to protocol revenue, and it scales with the share of takers who self-submit rather than use the app UI.

Likelihood

High

Recommendation

Bind the collected fee in onEvmSwapFunded and onTronFunded as onBtcFunded already does.

Medium Severity Issues

Zero-amount fund squats swapId and permanently blocks the leg

Resolved

Path

`SharedHTLC.sol`

Function Name

`SharedHTLC.fund`

Description

TRON has no cheap CREATE2 clone pattern, so SharedHTLC holds every swap for a given TRC-20 in a mapping(bytes32 => Swap) rather than in a per-swap contract. The API derives that key by stripping the dashes from the swap UUID and right-padding the result to 32 bytes, then embeds it in the unsigned fund, claim and refund triggers it hands to both parties. fund treats a non-zero initiator in the slot as proof that the swap already exists.

The key is caller-supplied and bound to nothing. fund never derives it from recipient, amount, hashlock or timelock, and never rejects amount == 0. Anyone holding the id calls fund with amount = 0 and any recipient, hashlock and timelock past the MIN_TIMELOCK floor; _transferFrom of zero succeeds on a standard TRC-20 with no balance and no approval, and the slot is written before the transfer runs, so the squat costs only transaction fees. Every later fund for that id reverts SwapExists. Neither claim nor refund clears the entry and the contract exposes no owner path to delete it, so the slot stays occupied for the life of the deployment and, because swap.id is immutable, the TRON leg of that swap can never be funded. This breaks the property the SwapExists guard is meant to express, that slot occupancy identifies the party funding the agreed terms; SharedHTLC treats occupancy as identity without ever binding the slot to those terms. HTLCFactory shows the intended shape: it derives its swapId on-chain as a hash over the encoded terms instead of trusting one from the caller.

The off-chain layer is not fooled by the squat. onTronFunded compares the on-chain recipient and amount against the agreed leg, so a zero-amount entry is rejected as a mismatch and the swap is never advanced. The damage is therefore liveness rather than accounting. Reaching it requires knowledge of swap.id, which the API serves only on authenticated, party-scoped routes, so the practical attacker is the counterparty or anyone holding operator data; the zero-padded low 16 bytes narrow the key shape but the UUID retains far too much entropy to brute-force. The EVM leg is unaffected.

Zero-amount fund squats swapId and permanently blocks the leg

(continued)

Impact

The TRON leg of the targeted swap can never be funded for the life of the deployment, forcing the parties to re-quote under a fresh id and stranding an already-funded counter-leg until its timelock expires.

Likelihood

High

Recommendation

Derive swapId on-chain from a hash over the agreed terms instead of accepting it from the caller, matching the pattern HTLCFactory already uses, so a slot can only be occupied by a funding that matches those terms.

Reject an amount of zero in fund.

Medium Severity Issues

Expired RFQs stay fillable, handing makers a free perpetual option

Resolved

Path

`service.ts`

Function Name

`sweepExpiredRfqs and cancelRfq (no propagation); acceptTerms (no RFQ gate);
createSwapFromAgreement (writes agreed back)`

Description

An RFQ carries a TTL and a lifecycle: the sweeper runs each minute, moves any open or negotiating RFQ past its expiry to expired, and notifies the creator to recreate it, while `cancelRfq` lets the creator withdraw the intent outright. Once an RFQ reaches agreed it is terminal, since `cancelRfq`, `reopenRfq` and `deleteRfq` all reject that status and nothing in the codebase ever writes the filled status the schema declares.

Neither lifecycle transition reaches the negotiation threads hanging off the RFQ. The sweeper updates only the `rfqs` row, `cancelRfq` does the same, no sweeper exists for threads, and `myThreads` applies no RFQ-status filter, so the thread stays visible and open. `acceptTerms` gates solely on `thread.status`, reads the RFQ row only to resolve leg roles, and never consults its status or expiry; `createSwapFromAgreement` then writes agreed straight back over expired or cancelled. Worse, no acceptance is ever cleared by expiry, and only a fresh proposal resets the flags, so a creator who accepted before the TTL lapsed leaves `takerAccepted` set indefinitely. The maker can then fire their own acceptance at any later moment and the swap is created with no further action by the creator. This breaks the property the TTL exists to establish, that a quoted price stops being executable once it goes stale, and it converts a time-boxed quote into a free option the maker exercises only when the market has moved against the creator.

The same gap produces a second outcome from one intent. The expiry notification tells the creator to recreate, so a recreated RFQ fills as swap S2 while the original thread later fills as S1, leaving the creator prompted by the platform to settle two swaps for a single trade, and leaving the original RFQ pinned at agreed with cancellation, reopening and deletion all refused. Nothing on-chain binds the creator until they fund, so the loss lands only when they act on the platform's own settle prompt, which is the designed and expected response to it.

Expired RFQs stay fillable, handing makers a free perpetual option*(continued)***Impact**

The creator settles at a price the market has moved past, or funds two swaps for one intent, because the platform presents a stale acceptance as a live agreed deal and gives them no way to withdraw the RFQ afterwards.

Likelihood

Medium

Recommendation

Reject acceptTerms when the RFQ is not open or negotiating or its expiry has passed, and stop createSwapFromAgreement from writing agreed over a withdrawn or expired RFQ.

Medium Severity Issues

Dust output consumes the BTC dedup slot and strands the swap

Resolved

Path

```
watcher.ts  
service.ts
```

Function Name

```
startBtcWatcher  
onBtcFunded
```

Description

The BTC watcher polls each active leg's P2WSH address through Esplora and fires onFunded for the first buried transaction paying it. Deduplication lives in Redis so a restart does not re-fire an already-handled event, and the SADD is deliberately used to mark and decide in a single atomic step, which also keeps overlapping polls safe because the poll interval does not await the previous run.

The funding path keys that set on the address alone, watcher:btc:funded <address>, and evaluates the SADD inside the guard condition so the slot is consumed before onBtcFunded runs. onBtcFunded then compares the reported value against legAmount and returns on a shortfall without clearing anything, and no SREM exists anywhere in the codebase. The rejected event has therefore consumed the address's one and only funding notification, and every later transaction paying the same address returns SADD == 0 and is skipped. The spend path four lines below shows the intended shape, keying on \${address}:\${txid} so each transaction is handled once rather than each address. The property that breaks is that a leg's real funding must always be observable, and here a single rejected observation permanently retires the address.

Dust output consumes the BTC dedup slot and strands the swap

(continued)

The consequence reaches past the BTC leg. `onBtcFunded` is the only writer of the leg's fund column and the only route into `initiator_funded`, and `shortLegFunded` advances to `counterparty_funded` only from that status, so a consumed slot leaves the entire swap pinned at agreed with no operator override in the route surface. Several distinct events consume the slot: an attacker's dust payment, a wallet that splits the payment across several outputs in one transaction while the watcher reads only the first matching output, a payment split across two transactions where the first falls short, and an institutional caller who follows the fund route's instructions, which return only the HTLC address and amount while `onBtcFunded` also requires the same transaction to pay the treasury when the BTC leg carries the protocol fee. The adversarial variant requires the poison to bury before the genuine funding, so the attacker must know the P2WSH before it is funded; it is not on-chain until then and the API serves it only on party-scoped routes, which makes the counterparty the practical attacker. The output must also clear the dust relay threshold, roughly 330 satoshis for P2WSH, so a one-satoshi output would not propagate.

Impact

The victim's Bitcoin sits escrowed and unusable until the 24h CLTV lapses while the swap can never advance, at a cost to the attacker of one dust output and no committed capital.

Likelihood

Medium

Recommendation

Key the funding dedup on `$(address):${txid}` as the spend path already does, and compute the funded amount from the address's total confirmed balance rather than a single transaction's value, so split payments and top-ups are counted and a rejected observation cannot retire the address.

Keep the SADD before the handler rather than moving it after, since the poll loop does not await its previous run and post-handler marking would let overlapping polls fire `onFunded` twice.

Medium Severity Issues

Claim and refund sweep only the largest UTXO, silently abandoning the rest of a multi-UTXO escrow

Resolved

Path

```
packages/settlement-core/src/btc-privy.ts  
packages/web/lib/btc-claim.ts
```

Function Name

```
btcClaimViaPrivy, btcRefundViaPrivy (settlement-core); claimBtc, refundBtc  
(web)
```

Description

All four Bitcoin HTLC spenders fetch the full UTXO set at the P2WSH, reduce it to the single largest entry, and build a one-input, one-output transaction paying that UTXO's value minus a fixed fee. There is no loop over inputs and no summation. A P2WSH address holds a set of independently spendable UTXOs, each satisfiable by the same witness, so any escrow funded across more than one UTXO leaves every entry except the largest behind. The remainder is not surfaced, warned about, or retried, and the function returns the broadcast txid, which every client renders as a completed claim or refund.

The two paths differ in who bears the loss. On refund the leftover is the sender's own coin and each remaining UTXO is still past the same timelock, so it is recoverable by refunding again; the harm is that success is reported and the second attempt is never prompted. On claim the leftover is genuinely lost: after the recipient sweeps the largest UTXO and is told the leg settled, the counterparty refunds the remaining UTXO after the timelock with their own key, taking coin the recipient was entitled to. A multi-UTXO escrow arises from a wallet that splits the funding across outputs, or from the top-up path where a short or split fund is misread on the funding side, the leg still shows unfunded, and the user sends the balance in a second transaction.

Impact

Part of a multi-UTXO escrow is abandoned while the UI reports success; on the claim path the left-behind coin is permanently lost to the counterparty, on the refund path it is silently stranded until the sender refunds again.

Likelihood

Medium

Claim and refund sweep only the largest UTXO, silently abandoning the rest of a multi-UTXO escrow *(continued)***Recommendation**

In all four spenders, add every UTXO at the P2WSH as an input, sign each with the same witness template, and pay the summed value minus a vsize-derived fee to one output; this shares the dynamic-fee work with the fixed-fee finding, since a multi-input sweep needs a size-based fee rather than a flat amount.

Low Severity Issues

_fund escrows unverified amount, permanently locking fee-on-transfer swaps

Resolved

Path

```
HTLCFactory.sol  
HTLCImplementation.sol
```

Function Name

```
_fund()
```

Description

createSwap deploys one minimal-proxy clone per swap and escrows the leg atomically in the same transaction. The swap parameters, including amount, are appended to the clone's bytecode as solady CWIA immutable args and read back via LibClone.argsOnClone. There is deliberately no initialize() and no storage slot for any parameter, which is what makes the clone address deterministic and lets a counterparty verify the exact escrow address before funding their own leg.

When token is a fee-on-transfer, deflationary, or negatively-rebasing ERC-20, _fund calls SafeTransferLib.safeTransferFrom(token, msg.sender, clone, amount) at HTLCFactory.sol:128 and never compares the clone's delivered balance against amount. The clone ends up holding amount - fee while its immutable args still assert amount. Both settlement paths then read that stale value: claim at HTLCImplementation.sol:100 and refund at HTLCImplementation.sol:110 both call _pay(tkn, to, amt) with the CWIA amount, and SafeTransferLib.safeTransfer reverts when the balance is short. This breaks the invariant every exit path depends on, that a clone's token balance is at least its immutable amount until settlement, and unlike the Tron sibling the clone cannot be corrected after the fact because there is no writable amount.

The bug is confined to ERC-20 legs. The native path is guarded by if (msg.value != amount + fee) revert BadValue() at HTLCFactory.sol:123, so it cannot under-deliver. Reaching the bug requires an asset registry entry for a token that delivers less than requested, which the factory's stated asset-generic design explicitly invites. It does not apply to standard ERC-20s, and it does not apply to the Tron leg: SharedHTLC.fund measures the delivered balance at SharedHTLC.sol:101-104 and overwrites the stored amount with the measured value.

_fund escrows unverified amount, permanently locking fee-on-transfer swaps *(continued)*

Impact

The counterparty's entire ERC-20 leg is permanently locked in the clone, since the delivered amount - fee makes both claim and refund revert on the immutable amount, while the other side refunds their leg intact.

Likelihood

Low

Recommendation

Measure the clone's token balance before and after the escrow pull in _fund and revert when the delivered amount is below amount. Reverting is the only in-place remedy, since the clone's amount sits in immutable CWIA args and cannot be corrected the way SharedHTLC.fund corrects its stored value.

Low Severity Issues

encodeScriptNumber truncates post-2038 timelocks, voiding the CLTV refund guard

Resolved

Path

`chains/bitcoin/htlc-script.ts`

Function Name

`encodeScriptNumber`

Description

The BTC leg's refund branch is guarded by a CLTV operand carrying the agreed absolute timestamp. `deriveBtcHtlc` calls `buildHTLCScript`, which encodes that timestamp through `encodeScriptNumber` and pushes it ahead of `OP_CHECKLOCKTIMEVERIFY`. Because the P2WSH address commits to the entire redeem script, that encoded operand is precisely what both counterparties are relying on when they verify the escrow address before funding.

`encodeScriptNumber` accumulates bytes with `abs & 0xff` and `abs >>= 8`, and both operators coerce their operand to a signed 32-bit integer. For any timestamp at or above 2^{31} that coercion turns the value negative: the first iteration pushes `0x00`, the shift yields a negative number, and the while (`abs > 0`) loop exits after a single byte. The function returns a lone `0x00` in place of the correct five-byte encoding, `buildHTLCScript` emits it as a one-byte push, and the operand `OP_CHECKLOCKTIMEVERIFY` reads is numerically zero. Under consensus rules a zero operand satisfies CLTV against any `nLockTime` below the 500000000 height threshold, so the refund branch carries no time constraint whatsoever; under default relay policy the same push is a non-minimal numeric encoding and the refund spend is instead rejected as non-standard. The claim branch is unaffected either way, since the defective push sits inside the unexecuted `OP_ELSE` body. This breaks the property the P2WSH address is supposed to commit to, that the escrow encodes the agreed refund deadline, and the encoder reports success while committing to no deadline at all.

Timelocks are server-derived as the current time plus the chain registry's configured `timelockSeconds`, so with Bitcoin at 86400 the boundary is roughly twelve years out of reach today. It becomes live when the wall clock passes 2038-01-19, or immediately if an operator configures a Bitcoin `timelockSeconds` beyond that margin. Neither `buildHTLCScript` nor `validateBitcoinHTLCParams` imposes any upper bound on the timestamp, and the encoder truncates silently rather than throwing, so nothing anywhere in the derivation path surfaces the loss.

encodeScriptNumber truncates post-2038 timelocks, voiding the CLTV refund guard *(continued)*

Impact

Every BTC HTLC derived past the boundary escrows to an address whose refund branch has no enforceable deadline, letting the funder reclaim their principal at any point after funding.

Likelihood

Low

Recommendation

Replace the 32-bit & and >> operations in encodeScriptNumber with arithmetic that does not coerce to int32, so timestamps across the full nLockTime range encode to their correct five-byte form.

Reject any timelock that exceeds CLTV's five-byte operand limit in both buildHTLCScript and validateBitcoinHTLCParams, so a value the encoder cannot represent fails loudly rather than yielding a script that silently drops the deadline.

Low Severity Issues

Floating Solidity Compiler Pragmas Can Introduce Build and Deployment Risk

Resolved

Path

-

Function Name

-

Description

The contracts use a floating Solidity compiler pragma such as `^0.8.x` instead of pinning compilation to a specific compiler version.

A floating pragma allows the contracts to be compiled with any compatible compiler version within the specified range. Different Solidity compiler versions can introduce changes in compiler behavior, optimizations, generated bytecode, or security-related fixes.

As a result, the source code may produce different bytecode depending on the compiler version selected by the developer, CI pipeline, or deployment environment. This can make deployments less deterministic and complicate bytecode verification and reproducibility.

Impact

Using different compiler versions may result in different bytecode and potentially different runtime behavior. It can also cause inconsistencies between audited, tested, and deployed bytecode, increasing the risk that a deployment does not exactly match the version reviewed during the audit.

Likelihood

Low

Recommendation

Pin the Solidity compiler to the exact version used for testing and deployment.

Informational Severity Issues

USDT Issuer-Level Blacklisting / Freezing

Acknowledged

Path

```
HTLCImplementation.sol  
SharedHTLC.sol
```

Function Name

```
claim() / refund() / _pay() / _push()
```

Description

The protocol treats USDT as a standard transferable ERC-20/TRC-20 token, while USDT can enforce issuer-level address blacklisting/freezing. If the HTLC escrow address, recipient, or initiator becomes blacklisted by the USDT issuer, the corresponding token transfer can fail.

In the EVM implementation, `claim()` and `refund()` ultimately rely on `SafeTransferLib.safeTransfer()`, while the TRON implementation uses `_push()`. Neither implementation provides an alternative settlement or recovery path when the underlying USDT transfer is rejected.

For example, if a swap's recipient is blacklisted before `claim()`, the claim transaction will revert and the escrowed USDT cannot be released to that recipient. Similarly, if the initiator is blacklisted when the timelock expires, `refund()` can revert. If the escrow address itself is frozen/blacklisted, the escrow may become unable to transfer its USDT balance altogether.

Impact

USDT escrowed in an HTLC can become inaccessible if Tether freezes/blacklists a relevant address. Depending on the affected address and blacklist state, users may be unable to claim or refund their funds, potentially resulting in permanent loss of access to the escrowed USDT.

Likelihood

Low

Recommendation

Document the issuer-level freezing/blacklisting risk for supported centralized stablecoins.

Informational Severity Issues

No Recovery Mechanism for Accidentally Sent or Stuck Tokens

Acknowledged

Path

```
HTLCImplementation.sol  
SharedHTLC.sol
```

Function Name

```
HTLCImplementation._pay() / SharedHTLC._push()
```

Description

The contracts do not provide any recovery or rescue mechanism for tokens or native assets that become stuck outside the intended escrow accounting.

For EVM HTLC clones, the escrow contract can receive native ETH through `receive()`, while ERC-20 tokens can also be transferred directly to the clone without going through `HTLCFactory.createSwap()`. However, the clone can only release the immutable token and configured amount through `claim()` or `refund()`. There is no administrative or user-accessible function to recover unrelated ERC-20 tokens or excess ETH sent to the contract.

Similarly, `SharedHTLC` is permanently associated with a single TRC-20 token through the immutable token variable. Any different TRC-20 tokens accidentally transferred to the shared contract cannot be withdrawn because `_push()` always operates on the configured token.

Impact

Accidentally transferred tokens or excess funds can become permanently inaccessible, resulting in an irreversible loss of user or protocol funds.

This is particularly relevant for the shared TRON contract because an arbitrary TRC-20 token sent to the pool has no supported path for recovery.

Likelihood

Low

Recommendation

Add a carefully restricted recovery mechanism for unsupported tokens and accidentally transferred funds.

Informational Severity Issues

SharedHTLC Can Over-Credit Swaps With Hook/Rebasing Tokens

Resolved

Path

`SharedHTLC.sol`

Function Name

`SharedHTLC.fund()`

Description

SharedHTLC.fund() determines the amount received by measuring the contract's token balance before and after the external `_transferFrom()` call. It then adjusts the credited amount whenever `received != amount`.

For tokens with transfer hooks, rebasing behavior, or other non-standard balance mechanics, the balance may increase by more than the amount supplied by the current funder. Because the condition uses `!=` rather than only handling a shortfall (`received < amount`), an unexpectedly higher balance delta can cause the swap to be credited with more tokens than the user actually intended to fund.

Since SharedHTLC maintains multiple swaps in a shared token pool, an over-credited swap could potentially consume tokens belonging to other swaps when claimed.

Impact

A non-standard token could cause a swap to be credited with an incorrect amount, potentially resulting in excess tokens being paid from the shared pool and reducing funds available to other users.

Likelihood

Low

Recommendation

Only adjust the recorded amount when the actual received amount is less than the requested amount.

Informational Severity Issues

Missing recipient validation lets a mis-set swap leg burn or strand its escrow

Resolved

Path

```
SharedHTLC.sol  
HTLCFactory.sol  
HTLCImplementation.sol
```

Function Name

```
createSwap, fund
```

Description

Neither createSwap nor fund validates recipient, so a leg can be funded with recipient = address(0) (or, in the pool, address(this)). On claim the escrow is sent there: the native EVM path burns it via safeTransferETH(address(0)), while for USDT the transfer-to-zero reverts, leaving claim unreachable until the initiator recovers via refund. No attacker can set another party's recipient, so this is funder or coordinator misconfiguration, and there is no sweep to reverse a burn.

Impact

For USDT the mis-set leg's claim reverts and the initiator recovers the full amount via refund; only if the native-coin asset is enabled does a zero recipient burn the entire leg amount irreversibly with no recovery path.

Likelihood

Low

Recommendation

Validate recipient in both createSwap and fund and revert when it is address(0).

Additionally reject recipient == address(this) in SharedHTLC.fund and recipient == msg.sender at both entry points, so a settled swap always targets a distinct real counterparty.

Functional Tests

Some of the tests performed are mentioned below:

HTLCFactory.sol

- ✓ createSwap accepts `initiator` as a free parameter, bound to neither msg.sender nor any signed RFQ
- ✓ refund later pays that immutable initiator, so an on-behalf funder receives nothing back
- ✓ createSwap accepts recipient == address(0) and initiator == address(0) with zero validation
- ✓ createSwap enforces only the MIN_TIMELOCK floor – no ceiling, and no view of the BTC leg's T1
- ✓ an EVM-leg timelock of 48h (longer than a 24h BTC T1) is accepted, violating the asymmetric-timelock invariant
- ✓ a funder-chosen extreme timelock leaves refund still reverting a full year later
- ✓ the escrow leg transfers via an unmeasured safeTransferFrom while the clone's immutable amount is baked to the requested value
- ✓ a fee-on-transfer token therefore under-funds the clone, and both claim and refund revert permanently
- ✓ the fee leg is a second unmeasured transfer, so FeeCollected overstates what the treasury actually received
- ✓ fee == 0 is accepted with no floor, no percentage rule and no quote binding, silently bypassing collection
- ✓ the watcher-bound fields (recipient, amount, token, timelock) still match the quote while the fee is zeroed
- ✓ a non-zero fee with treasury unset reverts FeesDisabled – the guard protects the sink, never mandates a fee
- ✓ the fee sink is the stored treasury cached before transfer; no caller-supplied fee amount can redirect it (refuted)
- ✓ setTreasury reverts for a caller that is neither owner nor FEE_ADMIN_ROLE, and treasury is unchanged (refuted)
- ✓ a FEE_ADMIN_ROLE holder redirects the entire fee stream in one call, with no timelock or two-step delay
- ✓ a FEE_ADMIN_ROLE holder points treasury at a reverting contract, bricking every native fee-bearing createSwap

- ✓ `renounceOwnership` makes `revokeRoles` unreachable, leaving `FEE_ADMIN` irrevocable and still able to redirect fees
- ✓ a blacklisted (ERC-20) or ETH-rejecting (native) treasury takes the whole `createSwap` down – DoS on swap entry, not exit
- ✓ `computeAddress` predicts the deployed clone exactly, over a fuzzed (salt, secret, amount, recipient, initiator) space
- ✓ the clone address depends only on (implementation, args, salt) and not on fee
- ✓ reproducing a victim's (salt, args) from the mempool makes their `createSwap` revert `DeploymentFailed`, at zero fee
- ✓ `swapId = keccak256(args)` omits salt, so two clones at different salts share one identifier
- ✓ the predicted clone address can be blacklisted before deployment, pre-emptively bricking one chosen swap's funding
- ✓ the constructor accepts a codeless implementation address with no code check
- ✓ every clone of a codeless implementation funds successfully but no-ops claim/refund – all escrow stranded, no recovery
- ✓ an attacker funds a decoy clone matching the agreed 4-tuple under their own hashlock and initiator, byte-identical on every watched field
- ✓ no deadline or staleness guard: mempool inclusion delay silently shrinks the negotiated absolute T2 down toward the floor

HTLCImplementation.sol

- ✓ claim releases only on `sha256(abi.encodePacked(secret)) == hashlock`, roundtripped against the Bitcoin-side vector
- ✓ claim is permissionless but always pays the immutable recipient – the open write is harmless (refuted)
- ✓ revealing the secret cannot redirect the payee to the caller
- ✓ refund reverts for every `block.timestamp` below `timelock` and succeeds at the boundary with no unexpected revert
- ✓ claimed and refunded are one-way and mutually exclusive; a repeat claim or a late refund reverts `AlreadySettled`
- ✓ the same holds in the refund-first direction: refund then claim reverts, with no flag reset
- ✓ conservation holds – a clone disburses its escrow at most once, under an explicit falsification attempt
- ✓ the settle-once latch is written before the external call, so a recipient re-entering claim mid-payout reverts

- ✓ claim and refund windows overlap for all $t \geq \text{timelock}$ – both are live at once
- ✓ an initiator front-runs a late claim with refund, keeping this leg's escrow and the now-public secret
- ✓ a fee-on-transfer under-funded clone has both exits reverting, with no owner, initializer, sweep or rescue to reach
- ✓ a blacklist of the clone (escrow) address itself bricks claim and refund simultaneously, with no recovery path
- ✓ a blacklisted or ETH-rejecting initiator bricks refund forever; the mirror case on recipient bricks claim forever
- ✓ a native claim to recipient $== \text{address}(0)$ succeeds and permanently burns the escrow
- ✓ a native refund to initiator $== \text{address}(0)$ burns it the same way, with no revert and no signal
- ✓ control: a swap with a valid recipient receives the full amount, isolating the zero-address burn
- ✓ ERC-20 donation or native overpay to a live clone is stranded – `_pay` only ever moves the immutable amount
- ✓ ETH attached to a payable admin function is permanently locked
- ✓ the raw un-cloned implementation decodes its own runtime bytecode as six phantom swap args
- ✓ native value reaching the raw implementation address cannot be recovered through any settlement path
- ✓ `_params()` on the bare implementation returns garbage instead of reverting
- ✓ a downward rebase of the escrowed asset leaves the clone holding less than its immutable amount and locks it
- ✓ the EVM leg has no emergency stop or killswitch at all, in contrast to the TRON pool's freeze
- ✓ the zero-amount boundary escrows nothing and cannot strand funds (refuted)
- ✓ the native `address(0)` sentinel routes consistently on both branches; the Slither constant-variable hit is an analysis artifact (refuted)

SharedHTLC.sol (TRON shared pool)

- ✓ `fund()` reads `received = balAfter - balBefore` straddling the external `transferFrom` with no reentrancy lock
- ✓ a sender-side transfer hook re-enters `fund()` for a DIFFERENT `swapId`, and the outer measurement absorbs both deposits
- ✓ the over-credited attacker then drains an innocent co-depositor, who can no longer claim or refund their principal

- ✓ re-entering with the SAME swapId reverts and does not inflate – the collision path is closed (negative control)
- ✓ the `!=` correction writes upward as well as downward, so an over-delivering (reflection/rebase-up) asset credits above principal
- ✓ control: a standard 1:1 token stores exactly the principal, isolating the direction of the defect
- ✓ a fix applying only the `<` clamp blocks the over-credit and leaves the victim whole
- ✓ a fix applying only the reentrancy guard blocks the nested fund – each mitigation is independently sufficient
- ✓ nothing enforces $\Sigma(\text{unsettled amount}) \leq \text{balanceOf}(\text{pool})$ after funding, and amounts are never re-measured
- ✓ on a balance-reducing token, early settlers take their full recorded amount and the last swap is permanently stranded
- ✓ the same desync reproduces via supply-cut / seize rather than rebase, with no pro-rata haircut anywhere
- ✓ the fee leg is unmeasured while the escrow leg is measured – within one function, one leg corrects for the skim and the other does not
- ✓ FeeCollected therefore overstates the treasury's real credit on a fee-on-transfer TRC-20
- ✓ swapId is a caller-supplied key bound to nothing, guarded only by first-writer-wins SwapExists
- ✓ amount == 0 is accepted, so squatting an agreed swapId costs the attacker zero tokens, zero allowance, zero fee
- ✓ the legitimate funder's fund() then reverts SwapExists permanently – that TRON leg can never be created
- ✓ the squatted slot carries the attacker's hashlock while matching the watcher's (recipient, amount, timelock), so the real secret fails with InvalidSecret
- ✓ initiator is never cleared after settlement, so a settled slot is un reusable for the pool's lifetime
- ✓ two distinct swapIds may carry the same hashlock and both settle under one secret – each still pays its own fixed recipient
- ✓ claim to recipient == address(0) transfers the pooled escrow to the zero address and burns it
- ✓ control: a valid recipient receives the full amount from the same pool
- ✓ a single blacklist of the pool contract address bricks claim AND refund for every swap simultaneously
- ✓ a blacklisted initiator bricks refund forever with no fallback recipient and no admin recovery

- ✓ a global token pause bricks refund while a recipient blacklist bricks claim – both exits are single-address pushes
- ✓ freeze() is a one-way latch on an immutable owner with no unfreeze and no ownership rotation
- ✓ after one freeze, new funding is permanently bricked with no on-chain path to recover control
- ✓ freeze does not release, re-route, or sweep any already-stuck escrow – it is not a recovery lever
- ✓ setTreasury is unvalidated: the owner can zero it (DoS the fee-bearing leg) or redirect the whole fee stream
- ✓ renouncing with treasury zeroed permanently bricks fee-bearing swaps
- ✓ a role-backup holder surviving renounce is the positive control for that brick
- ✓ the TRON leg accepts an extreme timelock with no ceiling, mirroring the EVM floor-only check
- ✓ a token whose transfer returns empty calldata and moves nothing is read as success, marking the swap settled without moving funds
- ✓ a codeless token address cannot be funded at all
- ✓ a returndata bomb makes the call consume unbounded gas, and a capped call bricks the path
- ✓ a balanceOf that fails blocks all new deposits into the pool
- ✓ _call and _balanceOf guard the zero-address token case asymmetrically, leaving a permanently dead pool
- ✓ a 4-byte transferFrom return hits an unguarded abi.decode and panics instead of reverting TransferFailed
- ✓ the balanceOf side is guarded against the same short return – the asymmetry is one-sided
- ✓ hashlock == 0 is accepted and produces a refund-only swap
- ✓ unbounded fee values are accepted at fund time with no ceiling
- ✓ initiator, recipient, hashlock and timelock are write-once after funding (refuted)
- ✓ setTreasury and freeze both reject any caller other than the immutable owner (refuted)

Bitcoin logic

- ✓ generatePreimageHashlock produces a 32-byte preimage and 32-byte sha256 hashlock that verify against each other
- ✓ a wrong preimage fails verification against the same hashlock
- ✓ the TS hashlock equals the sha256(abi.encodePacked(secret)) the EVM and TRON contracts check – one secret settles all legs

- ✓ the redeem script is well-formed: opens OP_IF, closes OP_ENDIF, and contains OP_SHA256 and OP_CHECKLOCKTIMEVERIFY
- ✓ P2WSH derivation is deterministic for a given script and yields a tb1(signet/testnet) address
- ✓ the claim witness selects the OP_IF branch with a minimal 0x01 data byte (BIP-147), not OP_1, and appends the redeem script last
- ✓ the refund witness pushes empty (FALSE) to take the OP_ELSE branch
- ✓ validateBitcoinHTLCParams rejects a 31-byte hashlock
- ✓ validateBitcoinHTLCParams rejects a below-dust amount
- ✓ extractPreimageFromWitness recovers the preimage from a claim witness and it hashes back to the hashlock
- ✓ extractPreimageFromWitness returns null for a refund witness – no preimage is published on the refund path

Settlement logic – settlement/*, chains/evm/*, chains/tron/*, chains/timelock.ts, rfq/service.ts

- ✓ resolveLegRoles puts the long leg on the slower chain (BTC) regardless of A/B position
- ✓ equal-tier chains break the tie toward leg B (the taker as initiator) and stretch the gap by MIN_TIMELOCK_GAP_SECONDS
- ✓ across every registry chain pair the long leg's timelock is strictly greater, by at least the minimum gap
- ✓ an unknown chain throws rather than silently defaulting a timelock
- ✓ legByChain, otherLeg, legChain and legHtlcAddress resolve correctly on a cross-family pair
- ✓ legByFamily returns leg A for BOTH legs when the two legs share a settlement family; legByChain collides the same way
- ✓ a same-chain pair passes requireSettleablePair today and creates a real swap – the collision is reachable now
- ✓ ethereum <-> base does not pass today because 'base' is absent from the registry (roadmap reachability, recorded not assumed)
- ✓ under the collision, leg B funds for real, is validated against leg A's terms, is rejected as a spoofed escrow, and is never recorded
- ✓ funding leg B first is dropped identically – the defect is order-independent
- ✓ control: a bitcoin/ethereum pair resolves both legs correctly, isolating the fault to family collision
- ✓ onEvmSwapFunded refuses to advance on any mismatch of recipient, amount, token or timelock against the agreed terms

- ✓ the TRON recipient comparison canonicalises hex and base58 to one encoding, so the same address is not falsely flagged as spoofed
- ✓ the funding-order invariant documented in shortLegFunded is a status label only – loadLeg checks party membership and nothing else
- ✓ the public v1 API issues short-leg funding calldata while the initiator's long leg is still empty
- ✓ the funding route never reads swap status – it issues calldata at any lifecycle point, including an already-settled swap
- ✓ the escrow is recorded, the status still reads `agreed`, and the counterparty's escrow address is handed to the attacker
- ✓ after the drain the system records a normal secret reveal and the counter-leg relay goes nowhere
- ✓ the gate that would have stopped this exists only in the browser, not in the API
- ✓ postQuote does reject an expired RFQ – the guard exists, in exactly one place
- ✓ a taker accept followed by expiry and an "expired" sweep notification still lets the maker accept later, creating a swap at the stale price
- ✓ cancelRfq does not stop it either – the explicit kill switch is not honoured on the accept path
- ✓ the window is unbounded: a 7-day-old expiry accepts identically, so no time check exists anywhere on that path
- ✓ rejectThread does still kill it – the stale option is revocable by the taker, if they know to
- ✓ computeProtocolFee returns zero at 0 bps (the current default) across small, zero and large amounts
- ✓ computeProtocolFee is exact from 1bps through MAX_FEE_BPS on a 6-decimal notional
- ✓ fractional fees floor rather than round, so the protocol never over-charges
- ✓ large bigint notionals (21M at 18 decimals) compute without precision loss
- ✓ negative, over-max and fractional bps are rejected, as are negative amounts
- ✓ feeBreakdown reports gross, fee and taker outlay with the fee charged on top, not deducted
- ✓ deriveFromEvmSignature recovers the EVM address and derives matching TRON base58 and BTC bech32 identities from one signature
- ✓ derivation is deterministic per key – a different message under the same key yields the same three addresses
- ✓ a different key yields different derived addresses

Threat Model

1. External Dependencies & Trust Boundaries

This section enumerates **everything the protocol does not fully control**.

1.1 External Dependencies Table

Note: the contracts use Solady, not OpenZeppelin, and SharedHTLC imports nothing at all (raw low-level calls), so there are no OZ rows.

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
LibClone (Solady)	Library	CWIA clone deployment (cloneDeterministic), address prediction (computeAddress), immutable-arg decoding (argsOnClone) in HTLCFactory / HTLCImplementation	Clone bytecode, args encoding, and CREATE2 derivation are correct and mutually consistent	Any layout or derivation mismatch breaks pre-funding address verification, the core security guarantee of the swap
SafeTransferLib (Solady)	Library	ERC-20 escrow pull and fee routing in _fund; claim/refund payouts; native transfers	Reverts on failed transfers; tolerates no-bool tokens (USDT)	Documented Solady caveat: no code-existence check on the token, so calls to a codeless address no-op "succeed" and a clone can appear funded while empty; mitigated only by off-chain asset allowlisting
OwnableRoles (Solady)	Base contract	Owner + FEE_ADMIN_ROLE gate setTreasury on HTLCFactory	Role bitmap logic correct; owner and fee-admin keys held securely	Compromised keys redirect future fees only; escrows are non-custodial, so blast radius is fee revenue. Single-step transferOwnership exists alongside the two-step handover

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
EVM escrow assets (native + ERC-20, USDT first)	External contracts	Assets escrowed per clone; address(0) is the native sentinel	Exact amount delivered on transferFrom; no fee-on-transfer or rebasing; transfers to recipient/initiator cannot be blocked	Fee-on-transfer under-delivers against the immutable CWIA amount with no correction path; USDT blacklist of clone, recipient, or initiator strands a leg asymmetrically
USDT-TRC20	External contract	Pooled escrow in SharedHTLC via low-level calls accepting empty-or-true returndata	Full amount delivered; returndata convention holds; no blacklisting of pool or parties	Pool is shared per token, so an under-delivering TRC-20 socializes the shortfall onto other users' escrows (acknowledged in code); blacklist bricks both claim and refund for affected swaps
bitcojnjs-lib / tiny-secp256k1 / ecpair / bip322-js	Libraries	P2WSH HTLC script construction, PSBT signing and serialization, key ops, BIP-322 auth	Correct script/witness encoding and secp256k1 arithmetic	Script-encoding bugs silently change the HTLC address or disable a spend branch. OP_SHA256 is defined over an arbitrary-length stack element, bounded only by the 520-byte element limit – there is no implicit size constraint. BOLT-3 offered/received HTLC outputs carry an explicit OP_SIZE 32 OP_EQUALVERIFY for the same cross-implementation reason.
viem	Library	EVM RPC client, account and tx construction, ABI encoding, keccak for swapId	Off-chain encoding matches the on-chain abi.encode layout	swapId or clone-address divergence between off-chain and on-chain defeats pre-funding verification

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
TronWeb	Library	TRON RPC and tx building against SharedHTLC	Correct TVM tx construction and base58/hex address handling	Address-format conversion errors bind escrow to the wrong recipient
hono / jose / zod (API layer)	Libraries	RFQ and negotiation endpoints, JWT session auth, input validation	JWT verification sound; schemas reject malformed input	Auth bypass lets an attacker act as a swap party; party membership alone gates sensitive mutations (cf. setSwapAddress), so identity integrity is load-bearing
Esplora API	External service	Sole BTC chain view: address txs, tip height, tx hex, broadcast for watcher and settlement	Honest, available, timely chain data	A compromised or lagging instance can hide or fabricate funding events the coordinator acts on; it cannot move funds, but it can desync the settlement state machine and burn claim windows
EVM / TRON RPC endpoints	External service	Watchers read chain state; keeper submits claim/refund txs	Honest chain view; availability inside timelock windows	Censoring or stalled RPC converts a liveness delay into fund loss (claim missed before the counterparty refund matures)
Redis (ioredis) + BullMQ	Infrastructure	One-shot SADD dedup of BTC funded/spent events; settlement job queue	Persistent and atomic; at-least-once job delivery	Data loss replays handled events; address-only FUNDED_KEY dedup is the verified dust-preemption vector; under the off-chain-compromise invariant a hostile Redis can suppress or force handler execution

Dependency	Type	How It Is Used	Trust Assumption	Associated Risks
PostgreSQL (drizzle-orm)	Infrastructure	State of record for RFQ, negotiation, and swap legs; drives keeper decisions	Row integrity; writes only via the coordinator	Tampered payout addresses or timelocks steer keeper actions; the confinement invariant must hold: even then, funds cannot leave the pre-committed on-chain receiver
Operational keys (factory owner, FEE_ADMIN, SharedHTLC owner, keeper keys)	Key material	Fee-sink rotation, TRON pool freeze/rotation, automated claim/refund submission	Keys uncompromised and available	Factory keys: fee theft only. SharedHTLC owner is immutable, so key loss permanently disables freeze, rotation, and treasury updates. Keeper compromise is confined by design: claim pays only the immutable recipient

2. Entry & Exit Points Analysis (Function-Level)

This is the **core of the threat model**. Every externally callable function must appear here.

2.1 Function Entry / Exit Table

Contract Name	Function	Category
HTLCFactory.sol	constructor(address impl, address treasury_)	What this function can do Pin the immutable implementation address, set the deployer as owner, set the initial treasury (address(0) allowed, fees start disabled) What this function cannot/should not do Run again after deployment; validate that impl is a correct HTLCImplementation (a wrong address bricks all future clones) Main invariant(s) implementation is immutable for the factory's lifetime Access level Implicit restriction (once, at deployment)

Contract Name	Function	Category
HTLCFactory.sol	createSwap(salt, hashlock, recipient, token, amount, timelock, initiator, fee)	What this function can do Deploy the CWIA clone at its deterministic address, escrow amount into it (msg.value for native, transferFrom for ERC-20), route the optional fee to the stored treasury, all in one transaction What this function cannot/should not do Alter any parameter after deployment (no initializer exists); accept timelock below now + 30 min; take a fee while treasury is unset; accept msg.value other than amount + fee (native) or nonzero msg.value (ERC-20); redeploy with the same salt and args Main invariant(s) Clone parameters are fixed at deploy; escrow reaches the clone in the same transaction; fee is on top of amount and never touches escrow Access level Public; initiator and recipient are caller-supplied, their binding to the agreed RFQ is enforced off-chain only

Contract Name	Function	Category
HTLCFactory.sol	computeAddress(salt, hashlock, recipient, token, amount, timelock, initiator)	What this function can do Predict the exact clone address for a salt and parameter set so a counterparty can verify the escrow address before funding their leg What this function cannot/should not do Change state; diverge from the address createSwap deploys for identical inputs Main invariant(s) Prediction equals deployment for identical inputs; this is the soundness of pre-funding verification Access level Public view
HTLCFactory.sol	setTreasury(address t)	What this function can do Rotate the protocol fee sink to any address, including address(0) to disable fees What this function cannot/should not do Touch escrowed funds, already-collected fees, or any deployed clone Main invariant(s) Only future fee routing changes; escrow custody is unaffected Access level Owner or FEE_ADMIN_ROLE holder

Contract Name	Function	Category
HTLCFactory.sol	Inherited OwnableRoles: transferOwnership, renounceOwnership, requestOwnershipHandover, completeOwnershipHandover, cancelOwnershipHandover, grantRoles, revokeRoles, renounceRoles	What this function can do Transfer or renounce ownership (single-step and two-step handover both exist); grant or revoke FEE_ADMIN_ROLE What this function cannot/should not do Reach escrowed funds; grant any capability beyond treasury rotation (no other role is wired to anything) Main invariant(s) Only the owner mutates roles; renouncing ownership with no fee admin permanently freezes treasury rotation Access level Owner (renounceRoles: the role holder themselves)

Contract Name	Function	Category
HTLCImplementation.sol	claim(bytes32 secret)	What this function can do Verify sha256(secret) against the immutable hashlock, mark the swap claimed, pay exactly amount to the immutable recipient, publish the secret via the Claimed event What this function cannot/should not do Send funds anywhere other than recipient regardless of caller; run after any settlement; accept a keccak256 preimage (hashlock is sha256 for Bitcoin Script compatibility) Main invariant(s) Settle-once (claimed/refunded flags, CEI); payout destination fixed in bytecode; the revealed secret is what unlocks the counterparty leg Access level Anyone holding the secret (keeper-submittable by design)

Contract Name	Function	Category
HTLCImplementation.sol	refund()	What this function can do After the immutable timelock, mark the swap refunded and pay exactly amount to the immutable initiator What this function cannot/should not do Run before timelock expiry; redirect funds; run after any settlement Main invariant(s) Mutually exclusive with claim; payout fixed to initiator; timelock is block.timestamp based Access level Anyone (payout target is fixed, so open callability is safe)
HTLCImplementation.sol	receive()	What this function can do Accept the native-coin escrow the factory forwards during createSwap What this function cannot/should not do Track direct transfers: native sent outside factory funding sits on top of amount and is unrecoverable, since settlement pays exactly amount Main invariant(s) Settlement amounts are read from immutable args, never from balance Access level Public payable

Contract Name	Function	Category
HTLCImplementation.sol	Views: hashlock, recipient, token, amount, timelock, initiator, claimed, refunded	What this function can do Expose the CWIA-decoded immutable parameters and the two settlement flags What this function cannot/should not do Mutate state; return different parameter values over time Main invariant(s) Parameters are decoded from the clone's bytecode tail, not storage, so they are constant for the clone's lifetime Access level Public view
SharedHTLC.sol	constructor(address token_, address treasury_)	What this function can do Pin the pool's TRC-20, set the deployer as owner, set the initial treasury What this function cannot/should not do Allow the owner to ever change (owner is immutable, no transfer path exists) Main invariant(s) One pool per token; owner fixed for the contract's life Access level Implicit restriction (once, at deployment)

Contract Name	Function	Category
SharedHTLC.sol	fund(swapId, recipient, amount, hashlock, timelock, fee)	What this function can do Occupy a free swapId slot, escrow the measured delivery into the shared pool (balance before/after), correct the stored amount to what was actually received, route the optional fee What this function cannot/should not do Reuse an occupied swapId; run while frozen; accept timelock below now + 30 min; take a fee with treasury unset. Should not, but currently does: accept a caller-supplied swapId bound to nothing on-chain, and amount == 0 Main invariant(s) Slot is occupied before any external call (re-entrant fund with the same swapId reverts); credited amount equals measured delivery, so no swap can claim more than it escrowed from the shared pool Access level Public

Contract Name	Function	Category
SharedHTLC.sol	claim(bytes32 swapId, bytes32 secret)	What this function can do Verify the sha256preimage against the swap's hashlock, mark it claimed, pay the stored amount to the recipient fixed at fund() What this function cannot/should not do Pay anyone other than the stored recipient; settle twice; operate on an unknown swapId Main invariant(s) Settle-once per swapId; recipient immutable after fund(); CEI ordering Access level Anyone holding the secret
SharedHTLC.sol	refund(bytes32 swapId)	What this function can do After the swap's timelock, mark it refunded and return the stored amount to the initiator What this function cannot/should not do Run before timelock expiry; run after settlement; redirect funds Main invariant(s) Mutually exclusive with claim; payout fixed to the funder recorded at fund() Access level Anyone

Contract Name	Function	Category
SharedHTLC.sol	freeze()	What this function can do Permanently stop new fund() calls ahead of a pool rotation What this function cannot/should not do Block claim or refund of existing swaps; be reversed (no unfreeze exists) Main invariant(s) In-flight swaps remain settleable after freeze; freeze is one-way Access level Owner only
SharedHTLC.sol	setTreasury(address t)	What this function can do Rotate the fee sink, including to address(0) to disable fees What this function cannot/should not do Touch pooled escrow or already-routed fees Main invariant(s) Affects future fee routing only Access level Owner only

Contract Name	Function	Category
SharedHTLC.sol	Views: swaps(bytes32), token, owner, treasury, frozen, MIN_TIMELOCK	What this function can do Expose full swap structs and pool configuration What this function cannot/should not do Mutate state Main invariant(s) swaps() is the on-chain source of truth the TRON watcher reconciles against Access level Public view

3. Asset Flow Mapping (Critical Contracts)

This section identifies **where money actually lives** and **how it moves**.

3.1 Asset-Holding Contracts

Contract	Asset Type	Custodied Assets
HTLCImplementation.sol (one clone per swap)	Native coin or ERC-20 (USDT first)	That single swap's escrowed amount, held from createSwap until claim pays recipient or refund pays initiator
SharedHTLC.sol	TRC-20 (USDT-TRC20)	Pooled escrow of all active TRON swaps for that token, one pool per token, held per swapId from fund() until claim or refund
BTC P2WSH HTLC address (script, not a contract)	Native BTC	The Bitcoin leg's escrow, held from funding until spend via the secret branch (claim) or the CLTV branch (refund)

3.2 Asset Entry & Exit Functions

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
HTLCFactory.sol	createSwap	Native (msg.value) or ERC-20 (transferFrom)	Escrow to clone + fee to treasury, same tx (factory holds nothing)	Public	Fee-on-transfer under-delivers vs the immutable CWIA amount with no correction path; Solady skips the token code-existence check; recipient/initiator are caller-supplied, RFQ binding is off-chain only
HTLCImplementation.sol	claim	None	Full amount to immutable recipient	Anyone with the secret	Secret goes public on-chain the moment it lands, starting the cross-chain race; recipient blacklisted by the token blocks claim until only refund remains

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
HTLCImplementation.sol	refund	None	Full amount to immutable initiator	Anyone, after timelock	Claim/refund race at the timelock boundary if the secret is revealed late; initiator blacklisted strands the escrow permanently (both exits blocked)
HTLCImplementation.sol	receive	Native	None	Public	Direct sends outside factory funding sit above amount and are unrecoverable; settlement pays from args, never balance
SharedHTLC.sol	fund	TRC-20 escrow to pool + fee to treasury	None	Public	swapId is caller-supplied and bound to nothing, and amount == 0 is accepted, so slot squatting can block the TRON leg; measured-delivery correction protects the pool but credits fee-on-transfer swaps below the agreed amount, breaking cross-chain 1:1
SharedHTLC.sol	claim	None	Stored amount to recipient fixed at fund()	Anyone with the secret	Pool is shared, so any accounting error pays out other swaps' escrow; recipient blacklist blocks the claim exit
SharedHTLC.sol	refund	None	Stored amount to initiator	Anyone, after timelock	Same shared-pool blast radius; initiator blacklist plus expired counterparty leg strands value
BTC P2WSH HTLC	funding output	BTC	None	Anyone (payer)	Funding value read from a single vout instead of summed outputs; a dust output consumes the address-only watcher dedup slot before the amount check runs

Contract	Function	Asset In	Asset Out	Caller	Risk Notes
BTC P2WSH HTLC	secret-branch spend	None	BTC to recipient key	Holder of the secret + recipient key	Spending reveals the secret in the witness, same race dynamics as EVM claim; spend-tx fee estimation eats into the payout. Every atom true: server wants the fee in the funding txid, web pays it in a second txid, TMA never pays it, the correct fundBtcLeg builder is uncalled, the UI says "one transaction," and the address-only dedup makes the strand un-completable.
BTC P2WSH HTLC	CLTV-branch spend	None	BTC to initiator key	Initiator key, after locktime	int32 encodeScriptNumber collapses timestamps at or above 2^{31} to 0x00, destroying the refund deadline for post-2038 locktimes. All four atoms true: EVM/TRON claim() has no upper time bound, htlc.refund has a consumer but no producer, BTC claim hardcoded 600 sats with no estimation, no RBF/bump/CPFP path. Registry confirms Bitcoin always gets the initiator's long leg. Correctly reframed from a race to a manufactured window.

Automated Tests

No major issues were found. Some false positive errors were reported by the tools. All the other issues have been categorized above according to their level of severity.

Closing Summary

In this report, we have considered the security of HashLock. We performed our audit according to the procedure described above.

A total of 17 issues were identified in HashLock, comprising 1 Critical, 5 High, 4 Medium, 3 Low, and 4 Informational severity findings.

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.

About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers. With seven years of expertise, we’ve secured over 1400 projects globally, averting over \$3 billion in losses.

Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project’s security with cutting-edge practices.



8+ Years of Expertise	1M+ Lines of Code Audited
50+ Chains Supported	1500+ Projects Secured

Follow Our Journey



AUDIT REPORT

September 2026

For



Hashlock Market



Canada, India, Singapore, UAE, UK

www.quillaudits.com audits@quillaudits.com